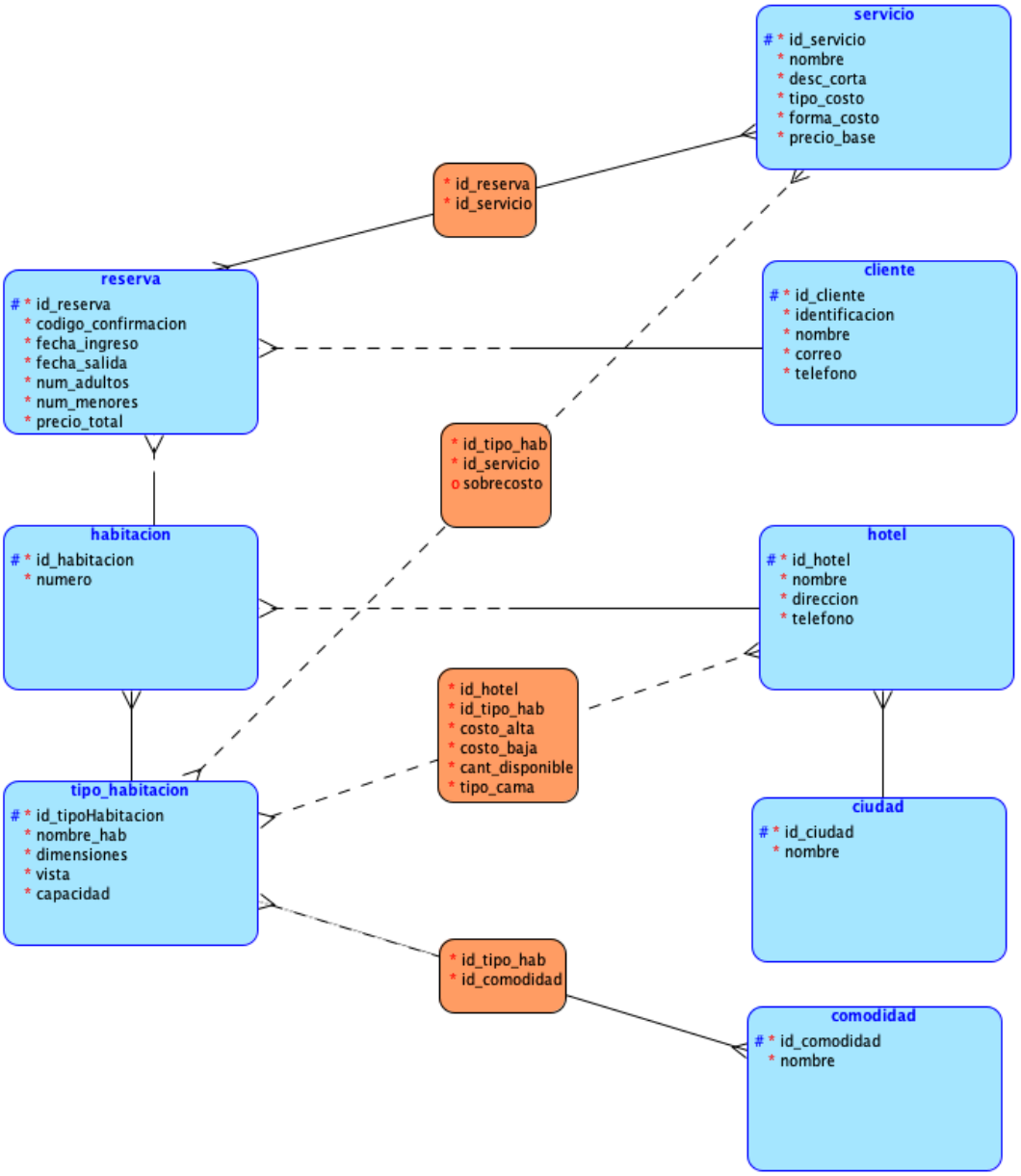
Modelo E/R:



Descripción del modelo relacional:

El modelo relacional está formado por varias tablas que representan las partes principales del sistema hotelero. Por ejemplo, hay una tabla para las ciudades, otra para los hoteles que pertenecen a esas ciudades y otra para los clientes que usan el sistema. También

están las tablas de tipo de habitación y habitación, donde se guarda la información de cómo son las habitaciones y a qué hotel pertenecen. La tabla de reservas guarda las reservas que hacen los clientes, con sus fechas y datos básicos. Además, existen tablas para las comodidades y los servicios que se pueden ofrecer. Para manejar las relaciones entre estos elementos, se usan algunas tablas intermedias que permiten conectar, por ejemplo, servicios con reservas o comodidades con tipos de habitación. En general, todas estas tablas trabajan juntas para organizar la información del sistema de manera clara y sin repetir datos innecesarios.

Universidad de los Andes

NORMALIZACIÓN

CAMBIOS:

Se realizaron ajustes puntuales al modelo relacional con base en la retroalimentación recibida, manteniendo la estructura general de las tablas ya definidas. El principal cambio consistió en modificar la entidad **RESERVA**, reemplazando el atributo **ID_HABITACION** por los atributos **ID_HOTEL** e **ID_TIPO_HAB**, con el fin de permitir un manejo adecuado de la disponibilidad por tipo de habitación en cada hotel. Adicionalmente, se incorporó el atributo **ID_HOTEL** en la tabla **HABITACION** para reflejar correctamente la relación entre habitaciones y hoteles. También se realizó un ajuste en el tipo de dato de uno de los atributos, pasando de **INTEGER** a **VARCHAR**, con el propósito de mejorar la consistencia en el manejo de identificadores. Estos cambios permitieron alinear el modelo con los requerimientos funcionales del sistema, evitando inconsistencias y manteniendo el cumplimiento de la Forma Normal de Boyce-Codd (BCNF).

DESCRIPCION:

El modelo relacional está formado por varias tablas que representan las partes principales del sistema hotelero. Por ejemplo, hay una tabla para las ciudades, otra para los hoteles que pertenezen a esas ciudades y otra para los clientes que usan el sistema. También están las tablas de tipo de habitación y habitación, donde se guarda la información de cómo son las habitaciones y a qué hotel pertenecen. La tabla de reservas guarda las reservas que hacen los clientes, con sus fechas y datos básicos. Además, existen tablas para las comodidades y los servicios que se pueden ofrecer. Para manejar las relaciones entre estos elementos, se usan algunas tablas intermedias que permiten conectar, por ejemplo, servicios con reservas o comodidades con tipos de habitación. En general, todas estas tablas trabajan juntas para organizar la información del sistema de manera clara y sin repetir datos innecesarios.

CUENTE

ID_CUENTE (VARCHAR)	IDENTIFICACION (VARCHAR)	NOMBRE (VARCHAR)	CORREO (VARCHAR)	TELEFONO(INTEGER)
PK, NA	NN, ND	NN	NN, ND	NN, ND
CL-001	10102020	Carlos Ruiz	carlos@mail.com	3189769034

TIPO_HABITACION

ID_TIPO_HAB (VARCHAR)	NOMBRE_HAB(VARCHAR)	DIMENSIONES (INTEGER)	VISTA (VARCHAR)	CAPACIDAD (INTEGER)
PK, NA	NN, ND	NN, CK(>0)	NN, CK(Mar, Piscina, etc)	NN, CK(>0)
TH-SJR	Suite Junior	45	Mar	4

COMODIDAD

ID_COMODIDAD (VARCHAR)	NOMBRE (VARCHAR)
PK, NN	NN
TD-5467	Calefacción

COMODIDAD_TIPO_HAB

ID_TIPO_HAB (VARCHAR)	ID_COMODIDAD (VARCHAR)
PK, FK	PK, FK
TH-SJR	TD-5467

HOTEL

ID_HOTEL (VARCHAR)	NOMBRE (VARCHAR)	DIRECCION (VARCHAR)	TELEFONO (VARCHAR)	ID_CIUDAD (VARCHAR)
PK, NA	NN	NN	NN	FK, NN
H-001	Dann-Alpes Royal	Calle 100 #15	601234567	C-BOG

CIUDAD

ID_CIUDAD (VARCHAR)	NOMBRE (VARCHAR)
PK, NA	NN, ND
C-BOG	Bogotá

HABITACION

ID_HABITACION (VARCHAR)	ID_TIPO_HAB (VARCHAR)	ID_HOTEL (VARCHAR)
PK	PK, FK	FK
HS-456	TH-SJR	HS-456

SERVICIO

ID_SERVICIO (VARCHAR)	NOMBRE(VARCHAR)	DESC_CORTA (VARCHAR)	TIPO_COSTO (VARCHAR)	FORMA_COSTO (VARCHAR)	PRECIO_BASE (INTEGER)
PK	NN	NULL	NN	NN	NN
S-DES	TH-SJR	50	Directo	Por_persona	100

RESERVA

ID_RESERVA (VARCHAR)	CODIGO_CONFIRMACION (VARCHAR)	FECHA_INGRESO(DATE)	FECHA_SALIDA(DATE)	NUM_ADULTOS(INTEGER)	NUM_MENORES(INTEGER)	PRECIO_TOTAL (INTEGER)	ID_CUENTE (VARCHAR)	ID_HOTEL (VARCHAR)	ID_TIPO_HAB (VARCHAR)
PK, NA	NN, ND	NN	CK(>FECHA_IN)	NN	NULL	NN	FK	FK	FK
R-999	XYZ123	10/05/26	15/05/26	2	2	500	CL-001	HS-456	TH-SJR

HOTEL_TIPO_HABITACION

ID_HOTEL(VARCHAR)	ID_TIPO_HAB (VARCHAR)	COSTO_ALTA (INTEGER)	COSTO_BAJA (INTEGER)	CANT_DISPONIBLE (INTEGER)	TIPO_CAMA (VARCHAR)
PK,FK	PK, FK	NN, CK(>0)	NN, CK(>0)	NN, CK(>=0)	NN
H-001	TH-SJR	550000	400000	15	King

SERVICIO_RESERVA

ID_RESERVA (VARCHAR)	ID_SERVICIO (VARCHAR)
PK, FK	PK,FK
R-999	S-DES

SERVICIO_TIPO_HABITACION

ID_TIPO_HAB (VARCHAR)	ID_SERVICIO (VARCHAR)	SOBRECOSTO(INTEGER)
PK, FK	PK,FK	NN
TH-SJR	S-DES	35

Desarrollo de la Aplicación:

Arquitectura de la Solución

El sistema está construido bajo una arquitectura web centrada en datos:

- **Capa de Datos:** Oracle Database 19c, responsable de la persistencia de información y del cumplimiento estricto de las restricciones de integridad (PK, FK, Check, Not Null) definidas en el modelo relacional.
- **Capa de Lógica y Presentación:** Oracle APEX, encargado de gestionar las reglas de negocio, procesar transacciones y proveer las interfaces CRUD para los usuarios administrativos y operativos.

Documentación de Módulos

Se desarrollaron módulos administrativos para la gestión de los datos maestros de la cadena hotelera Dann-Alpes. A continuación, se detallan las interfaces principales:

A. Gestión de Ciudades

Módulo central para el mantenimiento del catálogo geográfico. Su correcta parametrización es un prerrequisito estricto para el registro de nuevas sedes hoteleras, garantizando así la integridad referencial (Foreign Keys) en la base de datos al momento de expandir la cadena a nuevas ubicaciones.k

B. Gestión de Tipos de Habitación

Recently Created Tables	
DA_SERVICIO_TIPO_HABITACION	21 hours ago
DA_SERVICIO_RESERVA	21 hours ago
DA_RESERVA	21 hours ago
DA_HOTEL_TIPO_HABITACION	21 hours ago
DA_SERVICIO	21 hours ago
DA_HABITACION	21 hours ago
DA_COMODIDAD_TIPO_HAB	21 hours ago
DA_COMODIDAD	21 hours ago
DA_TIPO_HABITACION	21 hours ago
DA_HOTEL	21 hours ago
DA_CLIENTE	21 hours ago
DA_CIUDAD	21 hours ago

Interfaz destinada a la estructuración de la oferta de alojamiento. Permite registrar las categorías de habitaciones definiendo características operativas clave (capacidad, dimensiones y vista). El formulario asegura la completitud de la información antes de la inserción, respetando las restricciones de obligatoriedad (`NOT NULL`) del modelo relacional.

RFC1: Consultar Disponibilidad de Habitaciones

Objetivo del negocio: Conocer cuántas habitaciones de un tipo específico quedan disponibles en un hotel determinado para un rango de fechas, descontando el inventario base (CANT_DISPONIBLE) contra las reservas activas que se cruzan en dicho periodo temporal.

Go Actions

Hotel	Tipo Habitacion	Habitaciones Disponibles
Dann-Alpes Royal	Suite Junior	11

1 - 1

SELECT

```
  H.NOMBRE AS HOTEL,

  TH.NOMBRE_HAB AS TIPO_HABITACION,

  HTH.CANT_DISPONIBLE - COUNT(R.ID_RESERVA) AS HABITACIONES_DISPONIBLES

FROM DA_HOTEL_TIPO_HABITACION HTH

JOIN DA_HOTEL H ON H.ID_HOTEL = HTH.ID_HOTEL

JOIN DA_TIPO_HABITACION TH ON TH.ID_TIPO_HAB = HTH.ID_TIPO_HAB

LEFT JOIN DA_RESERVA R

  ON R.ID_HOTEL = HTH.ID_HOTEL

AND R.ID_TIPO_HAB = HTH.ID_TIPO_HAB

AND R.FECHA_INGRESO < DATE '2026-05-20'

AND R.FECHA_SALIDA > DATE '2026-05-01'

WHERE H.ID_HOTEL = 'H-001'

GROUP BY H.NOMBRE, TH.NOMBRE_HAB, HTH.CANT_DISPONIBLE;
```

Análisis de ejecución: Como se evidencia en la captura de pantalla tomada en Oracle APEX, la consulta utiliza un `LEFT JOIN` para asegurar que el hotel y tipo de habitación se muestren incluso si no tienen reservas asociadas. Al ejecutarla para el hotel 'H-001' y el periodo del 1 al 20 de mayo de 2026, el sistema contabiliza las reservas que interceptan esas fechas y las resta del inventario total, calculando una disponibilidad real de 11 habitaciones del tipo "Suite Junior".

RFC2: Consolidado de Reservas en el Tiempo

Objetivo del negocio: Generar un reporte estadístico que agrupa el volumen total de reservas realizadas, organizándolas por mes, ciudad y hotel. Este requerimiento permite identificar los picos de demanda y la ocupación histórica o futura de la cadena.

```
SELECT

  C.NOMBRE AS CIUDAD,
```

```
  H.NOMBRE AS HOTEL,
  TO_CHAR(R.FECHA_INGRESO, 'YYYY-MM') AS MES,
  COUNT(R.ID_RESERVA) AS TOTAL_RESERVAS
FROM DA_RESERVA R
JOIN DA_HOTEL H ON H.ID_HOTEL = R.ID_HOTEL
JOIN DA_CIUDAD C ON C.ID_CIUDAD = H.ID_CIUDAD
GROUP BY
  C.NOMBRE,
  H.NOMBRE,
  TO_CHAR(R.FECHA_INGRESO, 'YYYY-MM')
ORDER BY MES;
```

Ciudad	Hotel	Mes	Total Reservas
Barranquilla	Dann-Alpes Alto Prado	2026-05	3
Barranquilla	Dann-Alpes Barranquilla	2026-05	3
Medellín	Dann-Alpes Business	2026-05	3
Cali	Dann-Alpes Cali Centro	2026-05	4
Cartagena	Dann-Alpes Cartagena Centro	2026-05	3
Cartagena	Dann-Alpes Cartagena Mar	2026-05	3
Bogotá	Dann-Alpes Centro	2026-05	4
Bogotá	Dann-Alpes Ejecutivo	2026-05	3
Cali	Dann-Alpes Granada	2026-05	3
Medellín	Dann-Alpes Laureles	2026-05	4
Medellín	Dann-Alpes Medellín	2026-05	4
Bogotá	Dann-Alpes Norte	2026-05	4
Cartagena	Dann-Alpes Playa	2026-05	3
Bogotá	Dann-Alpes Premium	2026-05	3
Bogotá	Dann-Alpes Royal	2026-05	4

Análisis de ejecución: La ejecución de la consulta fue exitosa. A través de la función `TO_CHAR(FECHA_INGRESO, 'YYYY-MM')`, se logró extraer y agrupar el mes y año de las reservas insertadas por el script de población. La captura demuestra cómo el motor agrupa correctamente por ciudad y sucursal hotelera, mostrando el conteo final de transacciones comerciales proyectadas para mayo de 2026 en diferentes hoteles (ej. Dann-Alpes Laureles, Ejecutivo, Cartagena Mar, etc.).

RFC3: Flujo de Huéspedes por Ciudad y Hotel

Objetivo del negocio: Calcular el flujo diario de personas (sumando adultos y menores de edad) que ingresan a los hoteles de la cadena, segmentado por ciudad y fecha de ingreso. Esto es crucial para la planeación logística de servicios como el desayuno (restaurante) o limpieza.

```
SELECT
  C.NOMBRE AS CIUDAD,
  H.NOMBRE AS HOTEL,
  R.FECHA_INGRESO,
  SUM(R.NUM_ADULTOS + NVL(R.NUM_MENORES,0)) AS TOTAL_HUESPEDES
FROM DA_RESERVA R
JOIN DA_HOTEL H ON H.ID_HOTEL = R.ID_HOTEL
JOIN DA_CIUDAD C ON C.ID_CIUDAD = H.ID_CIUDAD
GROUP BY C.NOMBRE, H.NOMBRE, R.FECHA_INGRESO
ORDER BY R.FECHA_INGRESO;
```

Ciudad	Hotel	Fecha Ingreso	Total Huespedes
Barranquilla	Dann-Alpes Alto Prado	5/1/2026	6
Cali	Dann-Alpes Cali Centro	5/1/2026	6
Bogotá	Dann-Alpes Royal	5/1/2026	3
Bogotá	Dann-Alpes Centro	5/2/2026	6
Bogotá	Dann-Alpes Ejecutivo	5/2/2026	6
Cali	Dann-Alpes Granada	5/2/2026	3
Cartagena	Dann-Alpes Cartagena Mar	5/3/2026	3
Bogotá	Dann-Alpes Norte	5/3/2026	6
Bogotá	Dann-Alpes Premium	5/3/2026	6
Cartagena	Dann-Alpes Cartagena Centro	5/4/2026	3
Medellín	Dann-Alpes Medellín	5/4/2026	6
Cartagena	Dann-Alpes Playa	5/4/2026	6
Barranquilla	Dann-Alpes Barranquilla	5/5/2026	3

Análisis de ejecución: El código emplea de manera robusta la función `NVL` para prevenir que valores nulos en la columna `NUM_MENORES` anulen la suma aritmética del total de huéspedes. Al usar la función de agregación `SUM()`, se obtiene la carga exacta de visitantes que el hotel espera recibir por cada día específico en el calendario, organizado de manera cronológica ascendente.

RFC4: Ingresos Totales por Hotel y Ciudad

Objetivo del negocio: Proporcionar una visión financiera clara sobre el rendimiento económico de cada sucursal. Esta consulta permite a la gerencia identificar cuáles hoteles y ciudades están generando la mayor cantidad de ingresos brutos, facilitando la toma de decisiones sobre inversiones o promociones.

```
SELECT

  C.NOMBRE AS CIUDAD,

  H.NOMBRE AS HOTEL,

  SUM(R.PRECIO_TOTAL) AS INGRESO_TOTAL

FROM DA_RESERVA R

JOIN DA_HOTEL H ON H.ID_HOTEL = R.ID_HOTEL

JOIN DA_CIUDAD C ON C.ID_CIUDAD = H.ID_CIUDAD

GROUP BY C.NOMBRE, H.NOMBRE

ORDER BY INGRESO_TOTAL DESC;
```

Ciudad	Hotel	Ingreso Total
Bogotá	Dann-Alpes Royal	1790000
Cali	Dann-Alpes Cali Centro	1710000
Medellín	Dann-Alpes Laureles	1706000
Medellín	Dann-Alpes Medellín	1702000
Bogotá	Dann-Alpes Norte	1698000
Bogotá	Dann-Alpes Centro	1694000
Medellín	Dann-Alpes Business	1287000
Cartagena	Dann-Alpes Playa	1284000
Bogotá	Dann-Alpes Premium	1281000
Bogotá	Dann-Alpes Ejecutivo	1278000
Barranquilla	Dann-Alpes Alto Prado	1275000
Barranquilla	Dann-Alpes Barranquilla	1272000
Cartagena	Dann-Alpes Cartagena Centro	1269000

Análisis de ejecución: La consulta utiliza la función de agregación `SUM` sobre la columna `PRECIO_TOTAL` de la tabla `DA_RESERVA`. Gracias al triple `JOIN`, logramos asociar cada centavo recaudado con su hotel y ciudad correspondiente. El uso de `ORDER BY INGRESO_TOTAL DESC` es clave aquí, ya que prioriza en el reporte a los hoteles con mejor desempeño financiero. En los resultados obtenidos tras la población, se observa una distribución coherente de ingresos basada en las reservas generadas por el bloque PL/SQL.

RFC5: Servicios y Tipos de Habitación más Populares

Objetivo del negocio: Identificar las preferencias de consumo de los clientes. Al cruzar los tipos de habitación con los servicios adicionales solicitados, el hotel puede optimizar su oferta, por ejemplo, ofreciendo paquetes que incluyan los servicios más demandados para ciertos tipos de habitación.

```
SELECT

H.NOMBRE AS HOTEL,

TH.NOMBRE_HAB AS TIPO_HABITACION,

S.NOMBRE AS SERVICIO,

COUNT(SR.ID_SERVICIO) AS TOTAL_USOS

FROM DA_RESERVA R

JOIN DA_HOTEL H ON H.ID_HOTEL = R.ID_HOTEL
```

```
JOIN DA_TIPO_HABITACION TH ON TH.ID_TIPO_HAB = R.ID_TIPO_HAB

LEFT JOIN DA_SERVICIO_RESERVA SR ON SR.ID_RESERVA = R.ID_RESERVA

LEFT JOIN DA_SERVICIO S ON S.ID_SERVICIO = SR.ID_SERVICIO

GROUP BY H.NOMBRE, TH.NOMBRE_HAB, S.NOMBRE

ORDER BY TOTAL_USOS DESC;
```

Hotel	Tipo Habitacion	Servicio	Total Usos
Dann-Alpes Royal	Suite Junior	Desayuno	1
Dann-Alpes Cartagena Centro	Suite Junior		0
Dann-Alpes Ejecutivo	Suite Junior		0
Dann-Alpes Medellín	Suite Junior		0
Dann-Alpes Business	Suite Junior		0
Dann-Alpes Premium	Suite Junior		0
Dann-Alpes Royal	Suite Junior		0
Dann-Alpes Laureles	Suite Junior		0
Dann-Alpes Barranquilla	Suite Junior		0
Dann-Alpes Alto Prado	Suite Junior		0
Dann-Alpes Playa	Suite Junior		0
Dann-Alpes Norte	Suite Junior		0
Dann-Alpes Cartagena Mar	Suite Junior		0

Análisis de ejecución: Este requerimiento es técnicamente más complejo debido a que involucra una relación de muchos a muchos entre Reservas y Servicios (resuelta mediante la tabla intermedia `DA_SERVICIO_RESERVA`). Se optó por utilizar `LEFT JOIN` hacia los servicios para no excluir del reporte aquellas reservas que no contrataron servicios adicionales. La función `COUNT` nos permite medir la frecuencia de uso, lo que se traduce directamente en la "popularidad" de cada combinación de estancia y servicio extra.

Escenarios de Prueba

Pruebas de Calidad y Corrección en la Base de Datos

a. Pruebas de unicidad de tuplas (Primary Key / Unique)

Objetivo: Garantizar que no se puedan registrar dos entidades con el mismo identificador o valores marcados como únicos.

Caso Exitoso (Inserción de la tupla 1): Se registra un nuevo hotel con una llave primaria (`ID_HOTEL`) que no existe en el sistema.

```
INSERT INTO DA_HOTEL (ID_HOTEL, NOMBRE, DIRECCION, TELEFONO, ID_CIUDAD)
VALUES ('H-900', 'Dann-Alpes Prueba PK', 'Calle 100 # 10-10', '6019999999', 'C-BOG');
```

Results | Explain | Describe

1 row(s) inserted.

0.03 seconds

Caso de Falla (Inserción de la tupla 2): Se intenta ingresar otro hotel distinto, pero forzando el uso de la misma llave primaria (`H-900`) recién insertada.

```
INSERT INTO DA_HOTEL (ID_HOTEL, NOMBRE, DIRECCION, TELEFONO, ID_CIUDAD)
VALUES ('H-900', 'Dann-Alpes Duplicado', 'Calle Falsa 123', '6010000000', 'C-MED');
```

```
ORA-00001: unique constraint (WKSP_T2626.SYS_C00196364635) violated on table
WKSP_T2626.DA_HOTEL columns (ID_HOTEL)
ORA-03301: (ORA-00001 details) row with column values (ID_HOTEL:'H-900') already
exists
```

Resultado: El sistema protegió la integridad de la entidad denegando la transacción.

b. Pruebas de integridad referencial con FK (Foreign Key) *Objetivo:* Asegurar que todo registro "hijo" esté vinculado a un registro "padre" válido, evitando datos huérfanos.

Caso Exitoso: Se inserta un hotel asignándole una ciudad (`C-CAL`) que previamente existe en la tabla `DA_CIUDAD`.

```
INSERT INTO DA_HOTEL (ID_HOTEL, NOMBRE, DIRECCION, TELEFONO, ID_CIUDAD)
VALUES ('H-901', 'Dann-Alpes Cali Sur', 'Cra 80 # 10-15', '6023333333', 'C-CAL');
```

Results Explain Describe

1 row(s) inserted.

0.00 seconds

Caso de Falla: Se intenta registrar un hotel en una ciudad cuyo `ID_CIUDAD` (`C-XYZ`) no ha sido creado en el sistema.

```
INSERT INTO DA_HOTEL (ID_HOTEL, NOMBRE, DIRECCION, TELEFONO, ID_CIUDAD)
VALUES ('H-902', 'Dann-Alpes Inventado', 'Avenida Siempre Viva', '6015555555', 'C-XYZ');
```

```
ORA-02291: integrity constraint (WKSP_T2626.SYS_C00196364636) violated - parent key
not found
```

Resultado: El sistema exige que la dependencia exista antes de permitir el registro, validando la llave foránea.

c. Pruebas de integridad de acuerdo con restricciones de chequeo (Check) *Objetivo:* Validar que los datos ingresados cumplan con las reglas de negocio lógicas definidas a nivel de tabla.

Caso Exitoso: Se crea una reserva donde la fecha cronológica tiene sentido (el cliente sale del hotel días después de haber ingresado).

```
13  INSERT INTO DA_RESERVA (ID_RESERVA, CODIGO_CONFIRMACION, FECHA_INGRESO, FECHA_SALIDA, NUM_ADULTOS, NUM_MENORES, PRECIO_TOTAL, ID_CLIENTE, ID_HOTEL, ID_TIPO_HAB)
14  VALUES ('R-8000', 'CONF-OK-1', DATE '2026-08-01', DATE '2026-08-05', 2, 0, 800000, 'CL-001', 'H-001', 'TH-SJR');
```

Results Explain Describe

```
1 row(s) inserted.

0.02 seconds
```

Caso de Falla: Se intenta insertar una reserva donde la fecha de ingreso es posterior a la de salida, rompiendo la restricción `CHECK (FECHA_SALIDA > FECHA_INGRESO)`.

```
16  INSERT INTO DA_RESERVA (ID_RESERVA, CODIGO_CONFIRMACION, FECHA_INGRESO, FECHA_SALIDA, NUM_ADULTOS, NUM_MENORES, PRECIO_TOTAL, ID_CLIENTE, ID_HOTEL, ID_TIPO_HAB)
17  VALUES ('R-8001', 'CONF-BAD-2', DATE '2026-08-10', DATE '2026-08-05', 1, 0, 300000, 'CL-001', 'H-001', 'TH-SJR');
```

```
ORA-02290: check constraint (WKSP_T2626.SYS_C00196364716) violated
```

Resultado: Se previene la existencia de transacciones con inconsistencias temporales.

Balance del Plan de Pruebas

¿Qué se logró hacer?

Durante las pruebas se logró validar correctamente el funcionamiento de los principales requerimientos funcionales del sistema, incluyendo el registro de ciudades, hoteles, clientes, habitaciones y reservas. Asimismo, se comprobó que el comportamiento de las restricciones del modelo sea correcto, tales como llaves primarias, llaves foráneas, restricciones de unicidad y reglas de chequeo. Adicionalmente, se verificó que las consultas correspondientes a los requerimientos funcionales de consulta (RFC1 a RFC5) retornan resultados coherentes con los datos, permitiendo analizar información relevante como disponibilidad de habitaciones, flujo de huéspedes, reservas en el tiempo e ingresos por hotel.

¿Qué no se logró hacer?

No se realizaron pruebas de concurrencia, es decir, múltiples usuarios realizando búsquedas al mismo tiempo sobre la base de datos. Tampoco se implementó una manera automática para la actualización de la disponibilidad de habitaciones al momento de registrar una reserva, lo cual podría ser mejorado mediante una lógica adicional en la aplicación.

Conclusión

En general, el sistema cumple con los requerimientos establecidos, garantizando la integridad de los datos y permitiendo la correcta gestión de la información. Las pruebas realizadas evidencian que el modelo de datos es consistente y capaz de prevenir errores en la manipulación de la información.